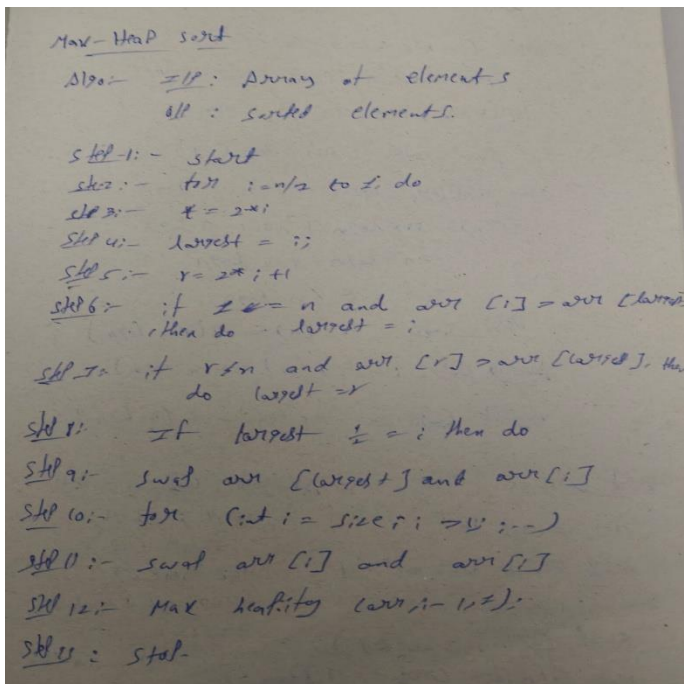


PRACTICAL-03

AIM: Implementation of max-heap sort algorithm.

1) MAX HEAP

ALGORITHM:



PROGRAM:

```
#include <iostream>

using namespace std;

void maxheapify(int arr[], int size, int i)
{
    int largest = i;
    int l = 2 * i;
    int r = 2 * i + 1;

    if (l <= size && arr[l] > arr[largest])
    {
        largest = l;
    }

    if (r <= size && arr[r] > arr[largest])
```

```
{
    largest = r;
}
if (largest != i)
{
    swap(arr[largest], arr[i]);
    maxheapify(arr, size, largest);
}
}

void heapsort(int arr[], int size)
{
    for (int i = size / 2; i >= 1; i--)
    {
        maxheapify(arr, size, i);
    }
    for (int i = size; i > 1; i--)
    {
        swap(arr[1], arr[i]);
        maxheapify(arr, i - 1, 1);
    }
}

int main()
{
    int n;
    cout << "Enter number of elements: ";
    cin >> n;
    int arr[n + 1];
    cout << "Enter elements: ";
    for (int i = 1; i <= n; i++)
    {
        cin >> arr[i];
    }
}
```

```

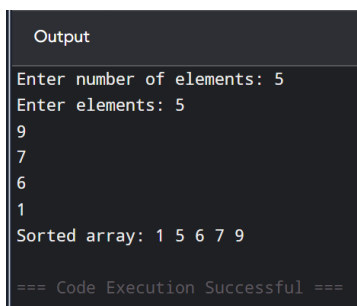
heapsort(arr, n);

cout << "Sorted array: ";

for (int i = 1; i <= n; i++)
{
    cout << arr[i] << " ";
}

return 0;
}
    
```

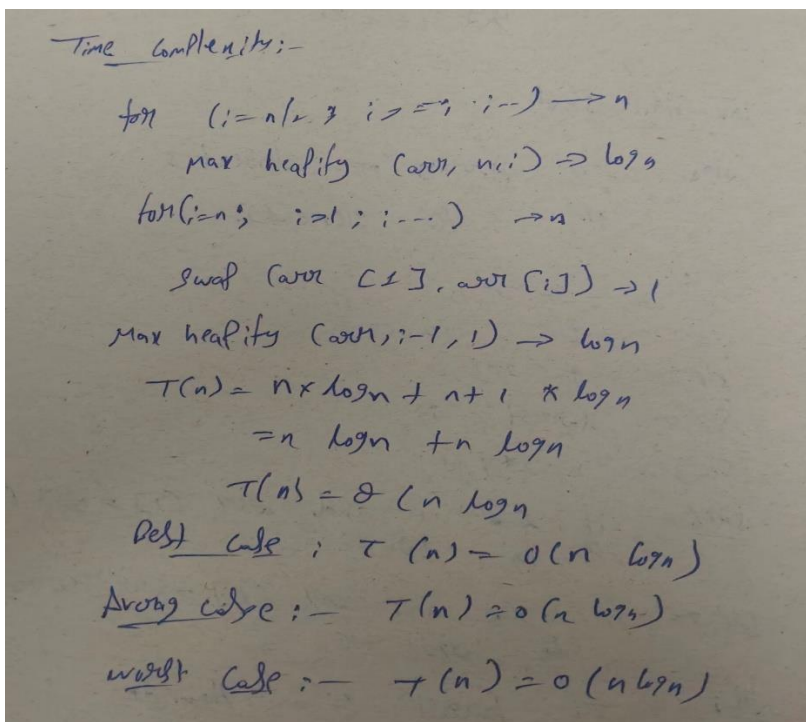
OUTPUT:



```

Output
Enter number of elements: 5
Enter elements: 5
9
7
6
1
Sorted array: 1 5 6 7 9
=== Code Execution Successful ===
    
```

TIME COMPLEXITY:



Time Complexity:-

for ($i = n/2$; $i >= 1$; $i--$) $\rightarrow n$
 max heapify (arr, n, i) $\rightarrow \log n$
 for ($i = n$; $i > 1$; $i--$) $\rightarrow n$
 swap (arr [1], arr [i]) $\rightarrow 1$
 max heapify (arr, i-1, 1) $\rightarrow \log n$
 $T(n) = n \times \log n + n + 1 \times \log n$
 $= n \log n + n \log n$
 $T(n) = O(n \log n)$
Best case : $T(n) = O(n \log n)$
Average case : $T(n) = O(n \log n)$
worst case : $T(n) = O(n \log n)$

Best Case :
for $i = n/2$ to 1
max heapify (arr, i) = n
for $i = n$ to 1
swap (arr[i], arr[1])
max heapify (arr, 1) // n
 $T(n) = n + n \times O(\log n)$
 $T(n) = O(n) + O(n \log n)$
 $T(n) = O(n \log n)$

Average Case $\rightarrow n$
for $i = n$ to 1
swap (i) $\rightarrow$
max heapify (i) -
 $T(n) = n + 1 + \log n$
 $T(n) = O(n) + O(n \log n)$
 $T(n) = O(n \log n)$

Worst Case :-
Max heapify from root to lowest level
of the heap $\Rightarrow \log n$
Max heapify () $\rightarrow \log n$
 $T(n) = n + n \times \log n$
 $T(n) = O(n \log n)$

Conclusion: Thus, the Max Heap Sort algorithm was successfully implemented. It efficiently sorts elements using a max heap with a time complexity of $O(n \log n)$.